

MUSTAFACODE

Lecture 15

Composition, Aggregation & Friend Functions (Detailed Notes)
Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

Lecture 15: Composition, Aggregation & Friend Functions (Detailed Notes)

Lecture No.15 — Composition, Aggregation & Friend Functions (Detailed Notes)

◆ 1. Composition

Definition

Jab ek class ke andar doosri class ka object hota hai to us relation ko:

Composition

kehte hain.

Yani:

"One object is part of another object"



Real Life Examples

Main Object	Sub Object
Car	Engine
House	Rooms
Student	Name(String)
Student	BirthDate(Date)



Example

```
class String{  
};  
  
class Student{  
  
    String name;  
  
};
```

Yahan:

```
Student has-a String
```

Isliye yeh Composition hai.

◆ 2. Student and String Example

String Class

```
class String{

private:
    char * ptr;

public:

    String(){
        ptr = NULL;
    }

    String(char * str){

        if(str != NULL){

            ptr = new char[strlen(str)+1];

            strcpy(ptr, str);
        }

        else{
            ptr = NULL;
        }

        cout << "Overloaded Constructor::String..\n";
    }

    ~String(){

        delete [] ptr;

        cout << "Destructor::String..\n";
    }
};
```

Student Class

```
class Student{  
  
private:  
  
    float gpa;  
    int rollNumber;  
  
    String name;  
  
public:  
  
    Student(char *, int, float);  
  
    ~Student();  
};
```

Student Constructor

```
Student::Student(char * n, int roll, float g)  
: name(n)  
  
{  
    cout << "Constructor::Student..\n";  
  
    rollNumber = roll;  
    gpa = g;  
}
```

Important Concept → Member Initialization List

```
: name(n)
```

isko:

Member Initialization List

kehte hain.

Yeh constructor body execute hone se pehle object initialize karta hai.

Main Function

```
int main(){  
  
    Student s1("Fakhir", 899, 3.5);  
  
    return 0;  
}
```

Output

```
Overloaded Constructor::String..  
Constructor::Student..  
  
Destructor::Student..  
Destructor::String..
```

Output Explanation

Step 1

```
name(n)
```

String object initialize hua.

Step 2

Student constructor execute hua.

Step 3

Program end par pehle Student destroy hua.

Step 4

Phir String object destroy hua.

◆ 3. Composition with Date Object

Ab hum Student class mein Date object add karte hain.

MUSTAFA CO



Date Class

```
class Date{

private:

    int day;
    int month;
    int year;

public:

    Date(){

        day = 1;
        month = 1;
        year = 2000;
    }

    Date(int d, int m, int y){

        day = d;
        month = m;
        year = y;

        cout << "Overloaded Constructor::Date..\n";
    }

    Date(const Date & obj){

        day = obj.day;
        month = obj.month;
        year = obj.year;

        cout << "Copy Constructor::Date..\n";
    }

    ~Date(){

        cout << "Destructor::Date..\n";
    }

};
```




Modified Student Class

```
class Student{

private:

    String name;
    Date birthDate;

    int rollNo;
    float gpa;

public:

    Student(char *, const Date &, int, float);

    ~Student();
};
```



Student Constructor

```
Student::Student(char * n,
                 const Date & d,
                 int roll,
                 float g)

: name(n), birthDate(d)

{
    cout << "Constructor::Student..\n";

    rollNo = roll;
    gpa = g;
}
```

Main Function

```
int main(){  
  
    Date d1(31,12,2000);  
  
    Student s1("Fakhir", d1, 899, 3.5);  
  
    return 0;  
}
```

Output

```
Overloaded Constructor::Date..  
Copy Constructor::Date..  
  
Overloaded Constructor::String..  
  
Constructor::Student..  
  
Destructor::Student..  
Destructor::Date..  
Destructor::String..  
Destructor::Date..
```

Important Point

Composition = Strong Relationship

Agar main object destroy ho jaye to andar wale objects bhi destroy ho jate hain.

Example:

```
Car → Engine  
Student → String  
Student → Date
```

◆ 4. Aggregation

Definition

Aggregation ek weak relationship hoti hai.

Ismein objects independently exist kar sakte hain.

Ek object doosre object ka pointer/reference rakhta hai.

Real Life Examples

Object 1	Object 2
Room	Chair
Teacher	Student
Bus	Passenger
Library	Books

Important Point

Composition mein object andar hota hai.

Aggregation mein sirf pointer/reference hota hai.

◆ 5. Room and Chair Example

Chair Class

```
class Chair{

public:

    Chair(){
        cout << "Chair Constructor\n";
    }

    bool FoldChair(){

        cout << "Chair Folded\n";

        return true;
    }

    bool UnFoldChair(){

        cout << "Chair UnFolded\n";

        return true;
    }

    ~Chair(){

        cout << "Chair Destructor\n";
    }
};
```



Room Class

```
class Room{

private:

    float area;

    Chair * chairs[50];

public:

    Room();

    void AddChair(Chair *, int);

    Chair * GetChair(int);

    bool FoldChair(int);

    ~Room();

};
```



Room Constructor

```
Room::Room(){

    for(int i=0; i<50; i++){

        chairs[i] = NULL;

    }

    cout << "Room Constructor\n";

}
```



AddChair Function

```
void Room::AddChair(Chair * chair1, int chairNo){  
  
    if(chairNo >=0 && chairNo <50){  
  
        chairs[chairNo] = chair1;  
    }  
}
```



FoldChair Function

```
bool Room::FoldChair(int chairNo){  
  
    if(chairNo >=0 && chairNo <50){  
  
        return chairs[chairNo]->FoldChair();  
    }  
  
    return false;  
}
```



Destructor

```
Room::~~Room(){  
  
    cout << "Room Destructor\n";  
}
```



Main Function

```
int main(){

    Chair ch1;

    {
        Room r1;

        r1.AddChair(&ch1,1);

        r1.FoldChair(1);
    }

    ch1.UnFoldChair();

    return 0;
}
```



Output Explanation

Step 1

Chair independently create hui.

Step 2

Room ne sirf chair ka address store kiya.

Step 3

Room destroy hua.

Lekin Chair object destroy nahi hua.

Step 4

Chair still usable thi:

```
ch1.UnFoldChair();
```



Aggregation = Weak Relationship

Objects independently exist karte hain.

◆ 6. Composition vs Aggregation

Composition	Aggregation
Strong Relationship	Weak Relationship
Object inside object	Pointer/Reference
Life dependent	Life independent
Destroy together	Separate destruction
Example: Car-Engine	Example: Room-Chair

◆ 7. Friend Functions



Definition

Jo function class ka member na ho lekin private members access kar sake usay:

Friend Function

kehte hain.

Why Needed?

Jab ek function ko multiple classes mein use karna ho.

Example Without Friend

```
class X{  
  
private:  
  
    int a,b;  
};
```

Global Function

```
void DoSomething(X obj){  
  
    obj.a = 3; // Error  
    obj.b = 4; // Error  
}
```

Why Error?

Kyuki private members sirf member functions access kar sakte hain.

◆ 8. Making Friend Function

```
class X{  
  
private:  
  
    int a,b;  
  
public:  
  
    friend void DoSomething(X);  
};
```



Friend Function Definition

```
void DoSomething(X obj){  
  
    obj.a = 3;  
  
    obj.b = 4;  
}
```

Ab koi error nahi aayega.



Important Rules

- ✓ Friend function class ke andar declare hota hai
 - ✓ Lekin member function nahi hota
 - ✓ friend keyword sirf declaration mein likhte hain
-

Wrong

```
friend void DoSomething(X obj){  
}
```

Correct

```
void DoSomething(X obj){  
}
```

9. Friend Classes

Ek class doosri class ki friend ban sakti hai.

Example

```
class X{  
  
    friend class Y;  
  
private:  
  
    int x;  
};
```

Class Y

```
class Y{  
  
public:  
  
    void SetValue(X obj){  
  
        obj.x = 10;  
    }  
};
```

Main Function

```
int main(){  
  
    X obj1;  
  
    Y obj2;  
  
    obj2.SetValue(obj1);  
  
    return 0;  
}
```

Important Point

Class Y ke member functions class X ke private members access kar sakte hain.

◆ 10. Real Life Examples

Concept	Real Example
Composition	Car has Engine
Aggregation	Room has Chairs
Friend Function	Helper function
Friend Class	Teacher accessing Student record

◆ 11. Interview Questions

Q1. Composition kya hoti hai?

Composition mein ek class doosri class ka object rakhti hai.

Q2. Aggregation kya hoti hai?

Aggregation mein ek class doosri class ka pointer/reference rakhti hai.

Q3. Main difference?

Composition strong relationship hai.

Aggregation weak relationship hai.

Q4. Friend function kya hota hai?

Aisa non-member function jo private members access kar sake.

Q5. Kya friend functions OOP violate karte hain?

Haan, kyuki encapsulation break hoti hai.

◆ 12. Final Summary

Composition

- Strong relationship
- Object inside object
- Dependent life

Example:

```
Student → String  
Car → Engine
```

Aggregation

- Weak relationship
- Pointer/reference
- Independent life

Example:

```
Room → Chair  
Bus → Passenger
```

Friend Function

- Non-member function
 - Private data access kar sakta hai
-

Friend Class

- Ek class doosri class ke private members access kar sakti hai
-

MUSTAFA CO